

Лабораторная работа № 5

«Естественная конвекция в полости»

Аннотация

Конвекция — один из способов передачи тепловой энергии. Она происходит при нагревании вещества (например, воздух или вода). Более теплая часть вещества становится менее плотной и поднимается вверх, а охладившаяся и более плотная опускается вниз, в результате возникают восходящие и нисходящие потоки вещества.

Цель данной работы – создать веб-приложение по запуску расчета в OpenFOAM задачи естественной конвекции в полости квадрата.

План занятия

- настройка файлов OpenFOAM;
- запуск решения в OpenFOAM;
- автоматизация удаления файлов прошлого решения;
- создание веб-приложения.

Форма представления выполненных заданий

Полученные в рамках данной лабораторной работы результаты прикладываются в качестве ответа в том же разделе LMS.

1 УСЛОВИЕ ЗАДАЧИ

Пример расчета будет взят из SOLIDWORKS FLOW SIMULATION 2018 (validations examples) 2018 г. пример: Natural Convection in a Square Cavity. По условию левая и правая стенки квадрата имеют температуру 305 К и 295 К соответственно. Верхняя и нижняя стенки адиабатические. Полость заполнена воздухом. Геометрия изображена на рисунке 1.

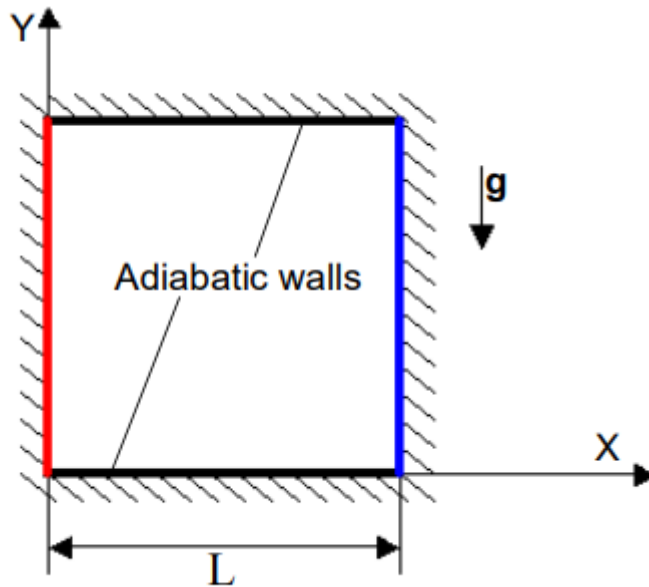


Рисунок 1 – Геометрия задачи

2 НАСТРОЙКА ФАЙЛОВ OPENFOAM

Для решения задачи будем использовать решатель buoyantSimpleFoam, который предназначен для вычисления теплового потока. Он находится в tutorials/heatTransfer/buoyantSimpleFoam. Копируем учебный пример buoyantCavity в свою рабочую директорию. Его возьмем за основу.

Удалите файлы, которые нам не понадобятся: Allclean, Allrun, README. В этой работе сосредоточимся на создании веб-приложения для автоматического запуска расчета, поэтому все файлы и папки для постпроцессной обработки удалим: папку validation и файл sample в каталоге system. Последний предназначен для извлечения указанных в файле полей для построения графика.

В папке constant убедимся, что в файле thermophysicalProperties указан идеальный газ в качестве среды. Остальные настройки не трогаем.

Листинг 1 – Файл thermophysicalProperties

```
thermoType
{
    type                heRhoThermo;
    mixture              pureMixture;
    transport            const;
    thermo               hConst;
    equationOfState      perfectGas;
    specie               specie;
    energy               sensibleEnthalpy;
}
```

Данный пример уже настроен для решения задачи конвективного течения воздуха в полости. Тип задачи турбулентный, используется модель kOmegaSST. Это можно посмотреть в каталоге constant в файле momentumTransport. Также в этом каталоге настраиваются и другие константы.

Нам необходимо изменить лишь температуру и расчетную область. Температуру меняем в файле 0/T. Согласно условию горячая стенка имеет температуру 305 К, а холодная стенка 295 К. Температуру полости установим, как среднюю: internalField uniform 300.

Листинг 2 – Изменение температуры

```
hot
{
    type    fixedValue;
    value    uniform 305;
}

cold
{
    type    fixedValue;
    value    uniform 295;
}
```

Теперь изменим размеры полости. Файл для настройки сетки находится в каталоге `system`. Открываем `blockMeshDict`. В файле присутствует строка: `convertToMeters 0.001`. Она означает, что все размеры будут переведены в метры путем умножения на заданный коэффициент. Поэтому размеры будем указывать в мм. Примем ширину и высоту полости 100 мм, толщину 10 мм. Сразу вынесем в отдельную переменную `b` размер полости. При обращении к переменной используется знак `$`. Внесем изменения в количество ячеек по каждому направлению (40 40 5).

Листинг 3 – Файл `blockMeshDict`

```
16 convertToMeters 0.001;
17 b 100;
18
19 vertices
20 (
21     (0 0 0)
22     ($b 0 0)
23     ($b $b 0)
24     (0 $b 0)
25     (0 0 10)
26     ($b 0 10)
27     ($b $b 10)
28     (0 $b 10)
29 );
30
31 blocks
32 (
33     hex (0 1 2 3 4 5 6 7) (40 40 5) simpleGrading (1 1 1)
34 );
```

Также внесем изменения в расположение горячей и холодной стенки. Слева будет расположена горячая, справа холодная.

Листинг 4 – Изменение расположения горячей и холодной стенки

```
cold
{
    type wall;
    faces
    (
        (6 5 1 2)
    );
}

hot
{
    type wall;
    faces
    (
        (4 7 3 0)
    );
}
```

Рассмотрим файл controlDict. Ключевое слово application задает решатель для выполнения расчета, в данном случае buoyantSimpleFoam. Начало расчета будет в startTime, которое равно 0. Записываться результаты будут через определенное количество шагов, об этом говорит параметр timeStep ключевого слова writeControl. Шаг (deltaT) равен 1. Конец решения будет через 1000 шагов, изменим данный параметр до 500. Запись результатов будет каждые 50 шагов (writeInterval). Ключевое слово purgeWrite задает количество записей. В данном случае будут сохраняться только три последние записи. Сохраняем изменения. Остальные файлы оставляем со стандартными настройками.

3 ЗАПУСК РЕШЕНИЯ

В терминале вводим команду для построения сетки – `blockMesh`. В каталоге `constant` должна появиться папка `polyMesh`. Просмотрите полученную сетку в `paraView`. Введите в терминале – `paraFoam`. В открывшемся окне программы нажмите на кнопку `Apply`, выберите `Solid Color` и `Surface With Color` в качестве отображения. Должна получиться сетка как на рисунке 2.

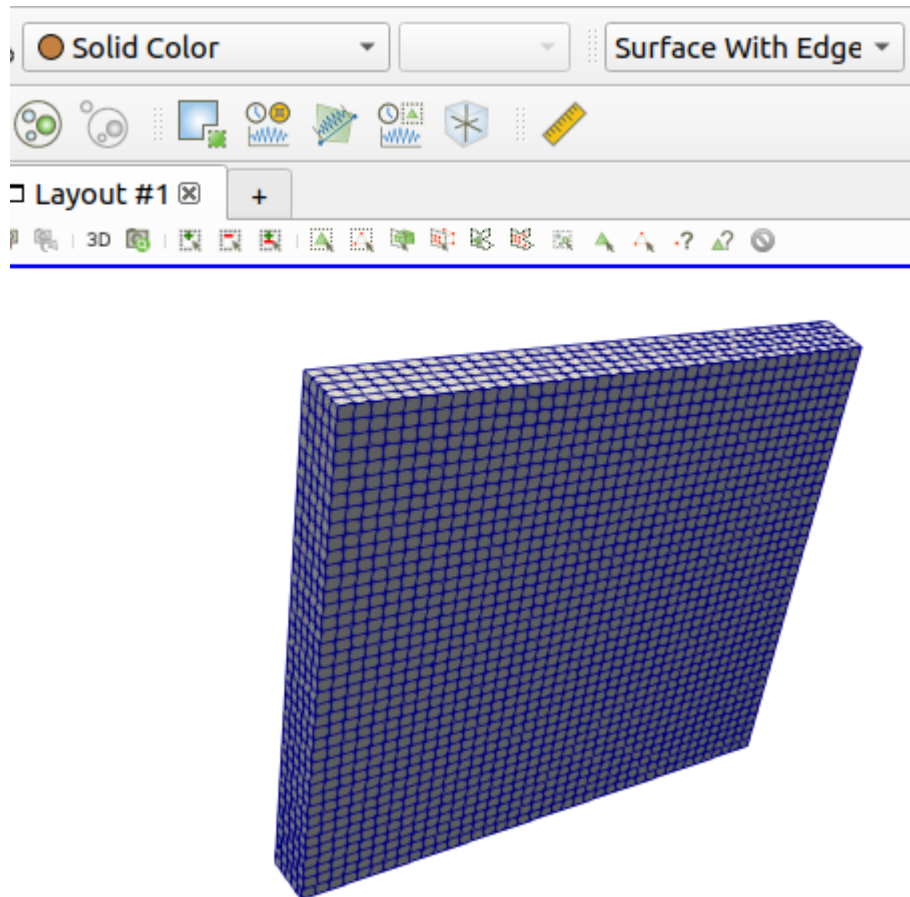


Рисунок 2 – Сетка в ParaView

Закройте окно `ParaView` и запустите решатель `buoyantSimpleFoam` в терминале. Дождитесь завершения выполнения расчета. После откройте окно `ParaView` снова для просмотра решения. На последний момент времени должен получиться результат как на рисунке 3. В качестве отображения выбираем температуру. Цвет фона можно изменить на панели `Properties` в разделе `Background`. Цветовую гамму измените на более привычную `rainbow` на панели `Properties` в разделе `Coloring`.

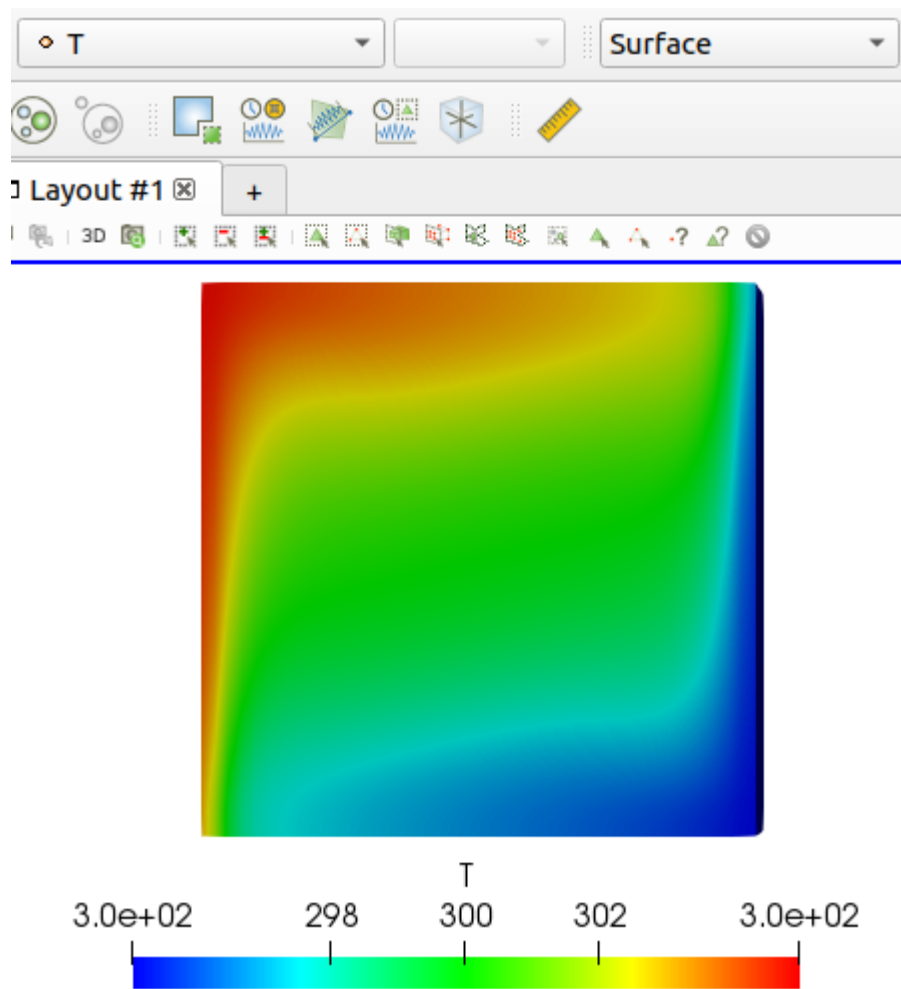


Рисунок 3 – Распределение температуры

4 АВТОМАТИЗАЦИЯ УДАЛЕНИЯ ПРОШЛОГО РЕШЕНИЯ

Создадим в папке решения новый файл Clean.sh, который будет отвечать за удаление файлов и папок прошлого решения. Командой `cd` осуществляется переход в папку с текущим примером. Вы вводите свой путь. Следующие команды `rm -rf` отвечают за удаление файлов решения. Сразу можно прописать все паки от 1 до 9, хотя сейчас в ходе решения создаются только папки 400, 450 и 500. Позже файл `controlDict` может быть изменен на запись решений в другие промежутки времени, и чтобы постоянно не изменять данный файл, сразу прописываем удаление всех возможных директорий. Команда `rm -rf log` необходима для удаления файла `log`, который будет создан веб приложением для вывода хода решения непосредственно в файл, а не терминал. Следующая команда удаляет сетку. По окончании выполнения в терминал выводится сообщение `clean`.

Листинг 5 – Файл Clean.sh

```
1 #!/bin/bash
2
3 cd /home/sveta/OpenFOAM/sveta-9/run/buoyantCavity/
4
5 rm -rf 1*
6 rm -rf 2*
7 rm -rf 3*
8 rm -rf 4*
9 rm -rf 5*
10 rm -rf 6*
11 rm -rf 7*
12 rm -rf 8*
13 rm -rf 9*
14 rm -rf log
15 rm -rf constant/polyMesh
16
17 echo clean
```

Протестируйте созданный файл. В терминале введите `./Clean.sh`.

5 СОЗДАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

5.1 Требуемые программы

Создайте каталог `cavitation`, в нем будет наше приложение. Расположение каталога выбираете самостоятельно. Код будет написан в программе Visual Studio Code. Убедитесь, что данная программа установлена у вас на компьютере. Если ее нет, то переходите на официальный сайт и выбирайте удобный способ установки.

Также для создания приложения потребуется установленная Node JS на компьютере, будем пользоваться пакетным менеджером `npm`. Убедитесь, что все установлено. Проверить можно с помощью команды: `node -v` и `npm -v`. Данные команды выводят версию `node` и `npm` соответственно. Если они завершили выполнение без ошибок и показали текущую версию, то можно приступать к созданию приложения. Иначе переходите на официальный сайт Node JS и `npm` и устанавливайте.

5.2 Создание файла конфигурации

Открываем в Visual Studio Code каталог `cavitation` с помощью `File => Open Folder`. Открываем терминал: `Terminal => New Terminal`. Вводим в нем команду для инициализации проекта: `npm init -y`. Флаг `-y` указывает на то, что файл создастся автоматически с дефолтными значениями без дополнительных вопросов. Если же вы хотите заполнить самостоятельно поля на данном этапе, то вводите команду без флага `-y`. Для создания приложения нам потребуются установить зависимости для разработки. Вводим в терминале: `npm i express body-parser`. Эта команда сразу установит два пакета. Также установим пакет, необходимый только на этапе разработки, `nodemon`. Он будет автоматически перезагружать сервер. Вводим в терминале: `npm i -D nodemon`. Установленный пакет должен появиться в разделе `devDependencies`. Главным файлом будет `index.js`, в разделе `scripts` установим скрипт для запуска приложения: `"dev": "nodemon index.js"`. Будем использовать `nodemon` для автоматического обновления.

Полученный файл package.json может отличаться версиями установленных пакетов. Сохраните полученный файл.

Листинг 6 – Файл package.json

```
{
  "name": "cavitation",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  > Debug
  "scripts": {
    "dev": "nodemon index.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "body-parser": "^1.19.1",
    "express": "^4.17.2"
  },
  "devDependencies": {
    "nodemon": "^2.0.15"
  }
}
```

5.3 Создание сервера

Теперь создадим файл index.js. Для начала необходимо создать сервер. Подключаем установленный модуль express. Запускать будем на порту 8000. Если данный порт у вас уже занят, укажите другой.

Листинг 7 – Создание сервера

```
var express = require('express')

var app = express()

app.listen(8000, function () {
  console.log('started at http://localhost:8000')
})
```

В консоли введите команду для запуска: npm run dev. Если получили сообщение: started at http://localhost:8000, значит сервер у вас запустился.

5.4 Создание html страницы

Теперь в корне проекта создаем основную html страницу. Для быстрого создания основных блоков воспользуемся Emmet. Введите восклицательный

знак в начале файла и нажмите tab на клавиатуре. Должен сформироваться шаблон страницы. В теге body создаем корневой div с классом container, который будет содержать один div с изображением полости и форму, в которой будут задаваться размер полости, температура холодной и горячей стенок и кнопка для расчета. Укажем атрибуты name для доступа к значениям, установим placeholder. Форме определим метод post.

Листинг 8 – Создание html разметки

```
<div class="container">
  <div class="card">
    <div class="card-image">
    </div>
    <form class="card-form" method='post' action="/">
      <div class="input">
        <input type="text" name="square" class="input-field"
          placeholder="Сторона квадрата (мм)" onKeyPress="handler()" required
        />
      </div>
      <div class="input">
        <input type="text" name="cold" class="input-field"
          placeholder="Температура холодной стенки (K)" onKeyPress="handler()" required
        />
      </div>
      <div class="input">
        <input type="text" name="hot" class="input-field"
          placeholder="Температура горячей стенки (K)" onKeyPress="handler()" required
        />
      </div>
      <div class="action">
        <input type="submit" class="action-button" id="btn" value="Расчет"></button>
      </div>
    </form>
  </div>
</div>
```

Элементам input установим событие handler на ввод значений. Вводить можно будет только цифры. Перед закрывающимся тегом body создаем тег script, который содержит функцию handler.

Листинг 9 – Создание функции handler

```
<script>
  function handler(){
    if (event.keyCode < 48 || event.keyCode > 57)
      event.returnValue= false;
  }
</script>
</body>
```

Подключаем стили. Создаем в корне директории файл style.css.

Листинг 10 – Подключение стилей

```
<link rel="stylesheet" type="text/css" href="style.css">
</head>
```

Вставляем в файл стили.

Листинг 11 – Файл style.css

```
@import url("https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;700&display=swap");

*,
*:after,
*:before {
  box-sizing: border-box;
}

body {
  font-family: "DM Sans", sans-serif;
  line-height: 1.5;
  background-color: #ffffff;
  padding: 0 1rem;
}

img {
  max-width: 100%;
  display: block;
}

input {
  appearance: none;
  border-radius: 0;
}

.card {
  margin: 2rem auto;
  display: flex;
  flex-direction: column;
  width: 100%;
  max-width: 425px;
  background-color: #fff;
  box-shadow: 0 10px 20px 0 rgba(#999, 0.25);
  padding: 0.75rem;
  border: 2px solid #eee;
}

.card-image {
  overflow: hidden;
  padding-bottom: 75%;
  background-image: url("convection.png");
  background-repeat: no-repeat;
  background-size: 100%;
  background-position: 0 15%;
  position: relative;
}

.input {
  display: flex;
  flex-direction: column-reverse;
  position: relative;
  padding-top: 1.5rem;
}
```

Листинг 12 – Файл style.css продолжение

```
.input-field {
  border: 0;
  z-index: 1;
  background-color: transparent;
  border-bottom: 2px solid #eee;
  font: inherit;
  font-size: 1.125rem;
  padding: 0.25rem 1rem;
}

.input-field:focus {
  color: #212529;
  outline: 0;
  box-shadow: 0px 1px 0px #053f5a;
}

.action {
  margin-top: 2rem;
}

.action-button {
  font: inherit;
  font-size: 1.25rem;
  padding: 0.5em;
  width: 100%;
  font-weight: 500;
  background-color: white;
  border: 1px solid #053f5a;
  border-radius: 6px;
  color: black;
}

.action-button:hover {
  box-shadow: 0 8px 16px 0 rgba(0, 0, 0, 0.2), 0 6px 20px 0 rgba(0, 0, 0, 0.19);
}

.action-button:active {
  transform: translateY(2px);
}
```

Добавьте в проект изображение полости. При необходимости внесите изменения в размеры отображения. Html страница должна получиться как на рисунке 4. Просто откройте файл в браузере.

Diagram of a square cavity with adiabatic walls. The side length is L . The coordinate system has x and y axes. Gravity g acts downwards.

Form fields:

- Сторона квадрата (мм)
- Температура холодной стенки (K)
- Температура горячей стенки (K)
- Расчет

Рисунок 4 – Интерфейс приложения

5.5 Создание функционала приложения

Откройте файл `index.js`. Добавим маршрутизацию приложению. По пути «/» приложение должно открывать созданную html страницу. Body-parser нужен будет для того, чтобы разбирать тела запросов, хранящиеся в `req.body`, объявим его на этом этапе тоже.

Листинг 13 – Создание маршрута

```

2  var bodyParser = require('body-parser')
3
4  var app = express()
5
6  const urlencodedParser = bodyParser.urlencoded({extended: false});
7
8  app.use(express.static(__dirname));
9
10 app.get('/', function (req, res) {
11   res.sendFile(__dirname + '/index.html')
12 })

```

Сохраните изменения. Перейдите на страницу `localhost` с соответствующим портом. Убедитесь, что html страница отображается.

Теперь объявим переменные, отвечающие за ширину полости, температуру горячей стенки, температуру холодной стенки, изменение параметров в файлах OpenFOAM, очистку предыдущего решения, запуск решения.

Листинг 14 – Объявление переменных

```
8 let square = '';
9 let hot = '';
10 let cold = '';
11 let changed = '';
12 let cleaned = '';
13 let runned = '';
```

Добавляем в код следующие строки, которые будут принимать введенные пользователем параметры, выводить их в консоль и присваивать переменным square, hot, cold соответствующие значения.

Листинг 15 – Получение параметров от пользователя

```
21 app.post('/', urlencodedParser, function (req, res) {
22   if(!req.body) return res.sendStatus(400);
23   console.log(req.body);
24   square = req.body.square;
25   hot = req.body.hot;
26   cold = req.body.cold;
27 });
```

Протестируйте. Введите в браузере любые значения, нажмите на кнопку «Расчет». Проверьте вывод в консоли vs code. Должен получиться объект с введенными вами значениями, например: { square: '90', cold: '298', hot: '304' }.

Теперь подключим exec для исполнения внешних функций. Будем вызывать исполнение bash скриптов.

Листинг 16 – Подключение exec

```
5 const { exec } = require('child_process');
```

Создаем первую функцию по чистке предыдущего решения. Создаем переменную script1, которая будет возвращать результат выполнения exec. Для запуска bash скрипта необходимо передать путь до него. Вы должны ввести свой путь. В консоль будет выведен результат выполнения функции, переменной cleaned будет присвоено соответствующее значение.

Листинг 17 – Функция удаления файлов предыдущего решения

```
31 ~ function cleanSh() {
32 ~   const script1 = exec('sh /home/sveta/OpenFOAM/sveta-9/run/buoyantCavity/Clean.sh',
33 ~     (error, stdout, stderr) => {
34 ~       console.log(stdout);
35 ~       console.log(stderr);
36 ~       if (error !== null) {
37 ~         console.log(`exec error: ${error}`);
38 ~       }
39 ~     });
40 ~   if (script1 == -1){
41 ~     cleaned = 'Не удалось очистить файлы предыдущего решения.'
42 ~     return cleaned
43 ~   }
44 ~   else {
45 ~     cleaned = 'Файлы предыдущего решения успешно удалены.'
46 ~     return cleaned
47 ~   }
48 ~ }
```

Далее необходимо внести изменения в файл температуры. Вынесите в отдельные переменные значения hot и cold. Измените значения value.

Листинг 18 – Изменение файла T

```
19 hot 305;
20 cold 295;
21
22 internalField uniform 300;

36     hot
37     {
38         type          fixedValue;
39         value          uniform $hot;
40     }
41
42     cold
43     {
44         type          fixedValue;
45         value          uniform $cold;
46     }
```

Теперь можно создавать функцию по изменению параметров в файлах решения OpenFOAM согласно введенным пользователем значениям. Константа middle отвечает за температуру в полости, она является средним значением введенных пользователем чисел.

Теперь в функции exec изначально нужно перейти в рабочую директорию с файлами решения. С помощью sed будем изменять значения на конкретных строках в файле 0/T и system/blockMeshDict. Если у вас переменные стоят на отличных от примера строках, то вводите в команду sed

номера своих строк. Результат выполнения будет выведен в консоль. Переменной `changed` будет присвоено значение согласно результату.

Листинг 19 – Создание функции изменения параметров

```
50 function changeParams(){
51     const middle = Math.round((+hot + +cold)/2)
52     const script = exec(`cd /home/sveta/OpenFOAM/sveta-9/run/buoyantCavity;;
53         sed -i "19s/hot .*/hot ${hot};;20s/cold .*/cold ${cold};;22s/internalField
54         uniform.*/internalField uniform ${middle};;" 0/T;
55         sed -i "17s/b .*/b ${square};;" system/blockMeshDict;
56         echo changed`,
57     (error, stdout, stderr) => {
58         console.log(stdout);
59         console.log(stderr);
60         if (error !== null) {
61             console.log(`exec error: ${error}`);
62         }
63     });
64     if (script == -1){
65         changed = 'Не удалось внести изменения в файлы.'
66         return changed
67     }
68     else {
69         changed = 'Файлы решения успешно изменены.'
70         return changed
71     }
72 }
```

Теперь осталось создать функцию по запуску расчета. Снова переходим в нужную директорию с помощью команды `cd`. Вызываем утилиту построения сетки, решатель `buoyantSimpleFoam`, результат будет записываться в файл `log`. Затем откроется `ParaView`. Установим размер буфера, чтобы обеспечить выполнение расчета. Результат будет выведен в консоль, переменной `runned` будет присвоено соответствующее значение.

Листинг 20 – Создание функции запуска решения

```
74 function runSh(){
75     const script2 = exec(`cd /home/sveta/OpenFOAM/sveta-9/run/buoyantCavity; blockMesh;
76         buoyantSimpleFoam >log; paraFoam;`, {maxBuffer: 1024 * 2000},
77     (error, stdout, stderr) => {
78         console.log(stdout);
79         console.log(stderr);
80         if (error !== null) {
81             console.log(`exec error: ${error}`);
82         }
83     });
84     if (script2 == -1){
85         runned = 'Не удалось запустить решатель.'
86         return runned
87     }
88     else {
89         runned = 'Решатель успешно запущен. Ожидайте завершения выполнения расчета. paraView откроется автоматически.'
90         return runned
91     }
92 }
```

Теперь добавляем в метод `post` блок `try-catch`. Последовательно вызываем созданные функции. В случае ошибки ответом будет служить

сообщение «Не удалось запустить расчет.», иначе произойдет переход на «/calc», который будет содержать страницу с ответом о запуске расчета.

Листинг 21 – Вызов функций при нажатии на кнопку «Расчет»

```
22  app.post('/', urlencodedParser, function (req, res) {
23      if(!req.body) return res.sendStatus(400);
24      console.log(req.body);
25      square = req.body.square;
26      hot = req.body.hot;
27      cold = req.body.cold;
28      try {
29          cleanSh()
30          changeParams()
31          runSh()
32          res.status(200)
33          res.redirect('/calc')
34      } catch (error) {
35          res.send('Не удалось запустить расчет.')
36      }
37  });
```

Создадим промежуточный роут result, в нем будем выводить в формате json результат запуска.

Листинг 22 – Создание дополнительного роута

```
22  app.get('/result', function (req, res) {
23      res.json({changedParams: `${changed}`, Cleaned: `${cleaned}`, Run: `${runned}`});
24  });
```

Протестируем полученный функционал. В браузере введите значения 90, 280, 300 для ширины, холодной стенки и горячей стенки соответственно. Нажмите на кнопку «Расчет». Должен осуществиться переход на «/calc». Пока эта страница еще не готова. Перейдите на «/result». У вас должен отобразиться ответ в формате json как на рисунке 5. Можно тестировать и с помощью программы Postman.

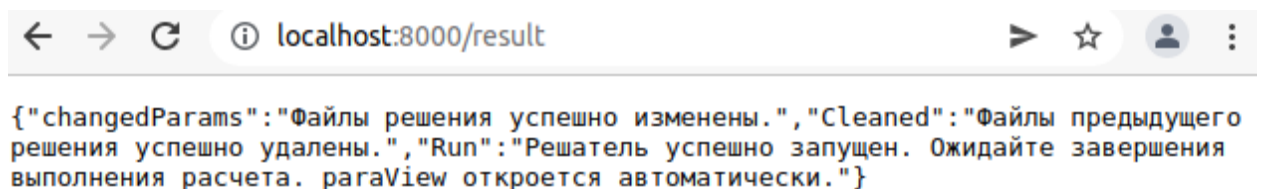


Рисунок 5 – Ответ в формате json

Также должен был запуститься расчет. Перейдите в каталог с файлами OpenFOAM. Проверьте, что расчет выполняется, должны создаваться папки с файлами решения, также должен был появиться файл log, который записывает

ход выполнения расчета. После некоторого времени должен открыться ParaView. Нажмите на кнопку Apply и проверьте результат расчета. Отобразите распределение температуры.

5.6 Создание html страницы с результатами

Осталось создать страницу, которая будет выводить результат запуска расчета. В корне проекта создаем файл calc.html. Вводим восклицательный знак, нажимаем tab. Должен сформироваться шаблон страницы. Тег body содержит в себе заголовок h1, div, в котором будет выводиться информация о запуске расчета, и заголовок h4, который просто сообщает путь, по которому можно посмотреть ход решения. Вы указывайте свой путь.

Листинг 23 – Разметка страницы calc.html

```
9  <body>
10  <h1 style="text-align: center">Расчет</h1>
11  <div id="block"></div>
12  <h4 style="text-align: center">Ход решения находится в файле log в каталоге
13  | /home/sveta/OpenFOAM/sveta-9/run/buoyantCavity/</h4>
```

Далее создаем стили для блока. Вы можете использовать свои стили.

Листинг 24 – Создание стилей

```
15  <style>
16  #block{
17  text-align: center; font-size: 16px;
18  width: 60%;
19  margin-left: 20%;
20  margin-bottom: 10px;
21  background: #f2f7f5;
22  padding: 1em;
23  color: #141313;
24  border-radius: 5px;
25  box-shadow: 0px 0px 2px 2px rgba(103, 99, 99, 0.7);
26  text-shadow: 0 1px 0 #615d5d;
27  border: 1px solid rgb(245, 237, 237);
28  }
29  </style>
30  </body>
```

Теперь создаем скрипт для получения результатов и вывода их в div. Объявляем url, с помощью fetch получаем данные и выводим в div с id = block. При возникновении ошибки, она будет выведена в консоль.

Листинг 25 – Вывод результата запуска расчета

```
30 <script type="text/javascript">
31   let url="http://localhost:8000/result";
32   fetch(url).then(response => response.json())
33   .then( (result) => {
34     console.log('success:', result)
35     let div=document.getElementById('block');
36     div.innerHTML=`<b style="font-size: 20px">Изменение параметров: </b><br>${result.changedParams}<br>
37     <b style="font-size: 20px">Очистка решения: </b><br>${result.Cleaned}<br>
38     <b style="font-size: 20px">Запуск решения: </b><br><span style="color: blue">${result.Run}</span>`;
39   })
40   .catch(error => console.log('error:', error));
41
42
43 </script>
```

Для того, чтобы страница отображалась, необходимо написать ее путь в файле index.js. Добавляем в файл следующие строки.

Листинг 26 – Доступ к странице с результатами

```
26 v app.get('/calc',(req,res)=>{
27   | res.sendFile(__dirname + '/calc.html')
28   |})
```

Тестируем приложение. Вводим в поля формы свои значения и нажимаем на кнопку «Расчет». Страница с результатами запуска должна выглядеть как на рисунке 6.

← → ↻ ⓘ localhost:8000/calc

Расчет

Изменение параметров:
Файлы решения успешно изменены.

Очистка решения:
Файлы предыдущего решения успешно удалены.

Запуск решения:
Решатель успешно запущен. Ожидайте завершения выполнения расчета. paraView откроется автоматически.

Ход решения находится в файле log в каталоге /home/sveta/OpenFOAM/sveta-9/run/buoyantCavity/

Рисунок 6 – Результат запуска расчета

ЗАКЛЮЧЕНИЕ

В ходе данной работы было создано веб приложение по автоматизации запуска расчета задачи естественной конвекции с помощью OpenFOAM. Программа была написана на Node.js. Были рассмотрены способы вызова bash скрипта для изменения значений в файлах решения, удаления предыдущих результатов расчета и запуска нового расчета с выводом хода решения в отдельный файл.

Вопросы для самоконтроля:

1. Что такое конвекция?
2. Каким способом можно запустить выполнение внешних функций bash?
3. Какое ключевое слово задает ограничение на количество временных каталогов?
4. В каком файле настраивается среда (в данном примере идеальный газ)?
5. Для чего используется пакет body-parser?